UNIVERSITY
OF HULL

Live Software Updates in a Real-Time System

Nathan Rignall

This dissertation is submitted on April, 2024
Word count: 2487

Abstract

My abstract ...

Acknowledgements

My acknowledgements ...

Contents

Chapter 1

Introduction

1.1 Project Background

1.2 Project Objectives

This project seeks to develop multiple methods and system architectures for live software updates in a real-time system via an example application. Using a timing analysis solution, such as a logic analyser, this will be tested to ensure that the real-time application meets all imposed deadlines (defined latency and jitter). The goal of this is to prove that it is feasibly possible to update software on a live system whilst maintaining real-time guarantees. Although this is a complex task the intention is to achieve this before February.

Another primary objective is to implement a reliable external testing method for a real-time environment, using existing research. This will allow for accurate measurement of the real-time behaviour of the program at all stages of the update, to validate that the upgrade is successful. Minimal external hardware is required, and the plan is to achieve this before February.

1.3 Technical Background

1.3.1 Real-Time

A real-time system is defined as a system which requires correct logical operation with completion in a configured deadline. This is in contrast to a non-real-time system which correct operation is desired but lacks configured deadlines. Kavi, Akl, and Hurson 2009

These real-time systems can be categorised into hard and soft. Hard real-time systems contain strict timing constraints, where a response must be calculated in a specified time-frame and a missed deadline could result in a catastrophic outcome. Soft real-time systems also have timing constraints, but a missed deadline should not result in a catastrophic outcome.

Typical hard real-time systems include aircraft or robotic control computers and soft real-time systems include media players or gaming computers. Soft real-time systems can be used in control applications, but the soft timing must be considered in the system design.

1.3.2 Control Systems

A control system Differential equation

1.3.3 Operating Systems

An Operating System (OS) is a piece of software that controls the execution of application programs on a system. Usually this involves managing resource allocations, such as a processor execution time, memory usage and generic inputs/outputs. Stallings 2018

In order to develop a real-time applications with an OS, a RTOS (Real-Time Operating System) is usually required. This is to ensure the time deadlines of an application are inforced.

Hard real time operating system are designed to run applications with absolute timing Worst-Case Execution Time Assigned a relative deadline, all tasks must complete in their deadline. Modelled mathematically to ensure that all tasks run to completion

Define the maxiumum delay after a deadline

Scheduling Memory

UNIX, POSIX

1.3.4 Mixed Criticality

1.3.5 Middleware

1.3.6 Software Updates

Chapter 2

Literature Review

2.1 Review

Analysis of existing dynamic software updating techniques for safe and secure industrial control systems

Challenges in Dynamic Software Updating

A survey of dynamic software updating

2.2 Case Study

2.2.1 Control Systems

Live Seamless Firmware Upgrade in Real Time Control Systems

2.2.2 Dynamic Updates

Dynamic Software Updates for Real-Time Systems.

2.2.3 Disruption-free

Disruption-free software updates in automation systems

2.2.4 A model

A Model for Updating Real-Time Applications

2.2.5 Grid-connected

Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics

2.3 Project Rational

Chapter 3

Requirements

3.1 Software Infrastructure

Embedded middleware is an intermediary abstraction layer that handles the interactions between applications and the underlying operating system/hardware (Noergaard, 2010). The project will develop the embedded software and the companion tools to support software updates that may take the form of middleware.

Framework for developers to run software components which can be dynamically updated During operation (define) Maybe some initial design too from the high level requirements

3.1.1 Functional Requirements

3.1.1.1 High-Level

REQ1 The software infrastructure must provide the functionality to run software components with a cyclic schedule.

- REQ1.1 The software infrastructure must allow users to create/start/stop/remove software components during operation.
- REQ1.2 The software infrastructure must allow users to change the configured cyclic schedule during operation.

REQ2 The software infrastructure must provide the functionality to allow components to communicate using messages passing.

- REQ2.1 The software infrastructure must allow users to configure communications between components using routes and channels.
- REQ2.2 The software infrastructure must allow users to add/remove routes between software components during operation.

REQ3 The software infrastructure must provide the functionality to save and restore state between components.

- REQ3.1 The software infrastructure must allow users to define which software components must save state during operation.
- REQ3.2 The software infrastructure must allow users to configure which component restore from state during operation.
- REQ3.3 The software infrastructure must allow users define state transformation maps so different versions of state can be restored.

3.1.1.2 Low-Level

REQ4 The software infrastructure must be configured using task lists.

REQ4.1 A task list must contain a series of actions with relevant data and is either non-blocking or blocking.

REQ4.2 A blocking action must complete in one execution cycle.

REQ4.3 All blocking actions in a task must complete in the same execution cycle.

REQ4.4 A non-blocking action can take multiple execution cycles to complete.

REQ5 A software component must be defined as a software template that developers can implement e.g. a Rust trait.

REQ5.1 Something to do with them being a process.

REQ6 The software infrastructure must implement the create component non-blocking action.

REQ6.1 The create component action must first load the software component binary from remote storage.

REQ6.2 The create component action must then start the software component binary as a separate process

REQ7 The software infrastructure must implement the start component blocking action.

REQ7.1 The start component action must mark the software component as ready to run.

REQ8 The software infrastructure must implement the stop component blocking action.

REQ8.1 The start component action must mark the software component as not ready to run.

REQ9 The software infrastructure must implement the add route blocking action.

REQ9.1 The add route action must add the relevant source and destination to the routing table.

REQ10 The software infrastructure must implement the remove route blocking action.

REQ10.1 The remove route action must remove the relevant source and destination from the routing table.

REQ11 The software infrastructure must implement the set schedule blocking action.

REQ11.1 The set schedule action must first check if the schedule is valid.

REQ11.2 Only components that are created can be added to the schedule.

REQ11.3 The schedule must define a execution frequency.

REQ11.4 The schedule must define which order components execute.

3.1.2 Non-Functional Requirements

3.2 Test Harness

To accurately test the real-time behaviour of the system, an external test harness is required. While latencies and jitter can be measured on-target with instrumentation, this does not independently measure the system's performance. The following requirements define the high-level functionality required for a successful test harness.

REQ12 The test harness must measure the timed latency and jitter between sending/receiving data to/from the software infrastructure.

- REQ12.1 The test harness must use independent software and hardware from the core infrastructure to verify the results.
- REQ12.2 The test harness must use a hard real-time platform to measure timings, for example a microprocessor.
- REQ12.3 The test harness must be paired with an analysis tool to visualize and analyze the measured data.

REQ13 An example software component must interface with the test harness using messages.

3.3 Demonstration Components

While the test harness is designed to evaluate the real-time behaviour of the system, additional software components are required to evaluate the correct behaviour of the system during update.

Example application that uses one differential equation and requires its state to be maintained with a plant for the example application to test it's functionality

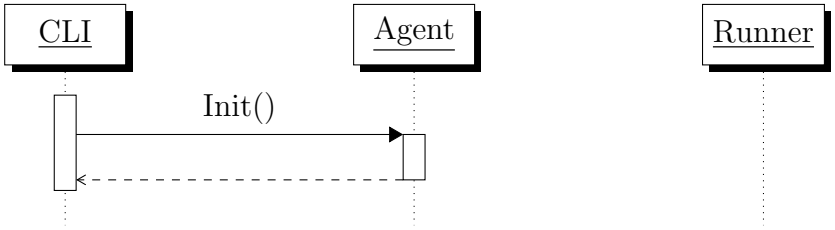
The example application must maintained

The plant must model a mass falling due to gravity

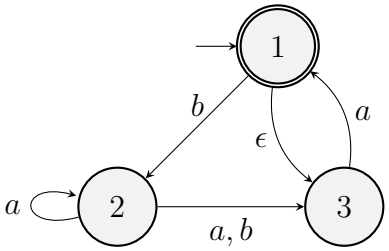
Chapter 4

Design

4.1 Operation



ClassName
name : attribute type name : attribute type = default value
name(parameter list) : type of value returned *name(parameters list) : type of value returned*



What is Linux Unix process model How containers work What is Xen

4.2 Runtime

To select the most optimal environment to develop the software infrastructure upon, a serieis of benchmarking activties were carried out. These focused on evaluating operating systems and hypervisors which could operate in a real time environment. The search was restricted to free and open source software which supports the Rasberry Pi Compute Module 4.

EXPLAIN THE VIRTUALISATION EXTENSIONS AND VIRTUALISATION

(Abeni and Faggioli 2020) compiled a similar study into the real time performance of open source hypervisors. Comparing KVM and Xen on various intel x86 processors.

SUMMARISE THE WORK HERE

For the purpose of this project the real-time performance on an Arm64 target, the chosen development target, must be specifically analysed. It is reasonable to expect that the performance characteristics may be different between the processor architectures due to the implementation of virtualisation extensions. This analysis shall compare the behaviour of 'bare metal' Linux and the Xen hypervisor, not including KVM.

This is because KVM is implemented ontop of the Linux kernel.

To reap the benefits of dynamic updates it is preferred that the hypervisor is independent of the operating system (explain why).

Also, in a true real-time system it is unusual to use linux. Rather a more preferable OS such as Zephyr or FreeRTOS (Free and open source solutions used.

Comparing straight linux and linux + Xen should give us a good method to estimate the specific latencies brough about by Xen.

EXPLAIN RT LINUX

Explain Yocto and Bitbake here too?

The RT-Tests suite includes a cyclictest program. This is a simple yet effective program that can be used to estimate the real time latencies in a system. It measures real time jitter by creating a interrupt event at a specific time in the future and measuring the time delay between the requested wake-up and actual wake-up time. While this provides a good method for identifying operating system induced latency and jitter it does not isolate

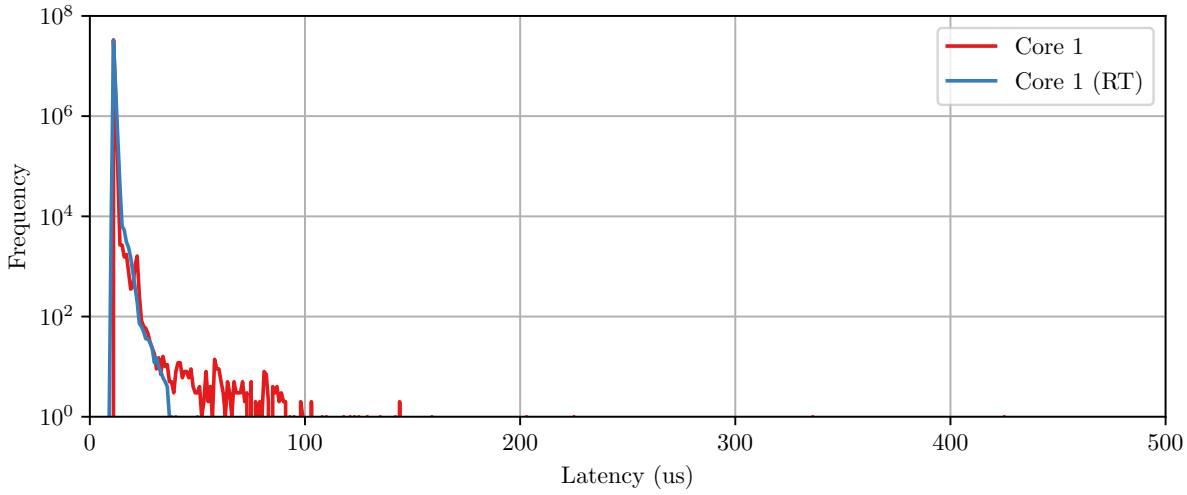


Figure 4.1: Latency Graph

latencies induced by hardware or low-level firmware. At this stage it can be assumed that the selected development targets do not induce significant latencies.

Under load rt-characterisics change

To set a baseline we shall first compare the real time behaviour of Linux with the standard Preemnt and Preempt RT kernels.

There is little difference between the two kernels when the system is mostly idle. However when the system is under load the differences between the kernel's schedulers becomes apparent. Especially so for memory stress versus cpu stress.

Now we are satisfied that linux, with the real time kernel patch performs adequately we can begin configuring Xen.

Exceptionally poor performance at first. Assume that is due to the nature of the Xen scheduler implementation.

EXPLAIN THE XEN SCHEDULER OPTIONS

When using the None scheduler these latencies and jitter are significantly improved

Multi core interference

The first experiments were performed on the linux kernel running without a hypervisor.

The next experiments used a real-time linux kernel.

Xen Scheduler (Credit vs)

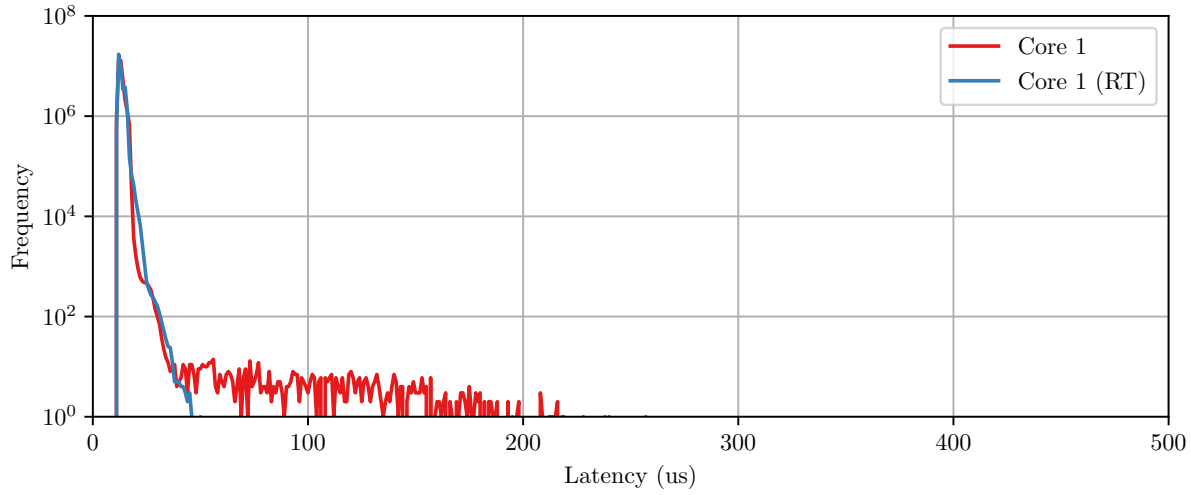


Figure 4.2: Latency Graph

4.3 Scheduler

To implement an event based scheduler it must be able to send signals between processes without significant jitter and latency. The level of jitter that is acceptable can be estimated by calculating a worst case execution time for a control loop in a particular application.

There are several different methods that can be used to send signals between processes in a Linux system. Unix shared memory and sockets shall both be investigated as they can be easily transported onto a non-linux POSIX system.

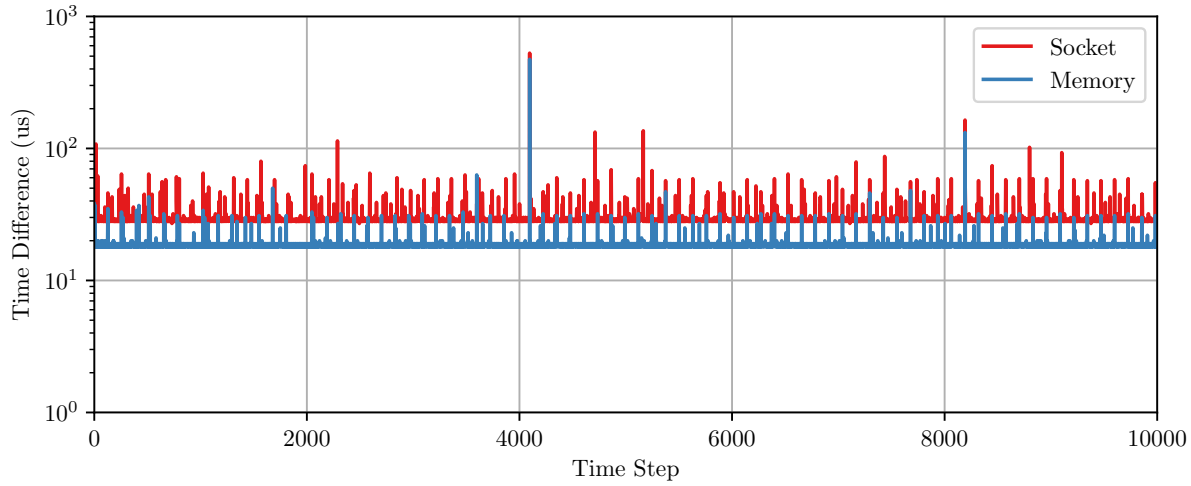


Figure 4.3: Compare Execution Latency

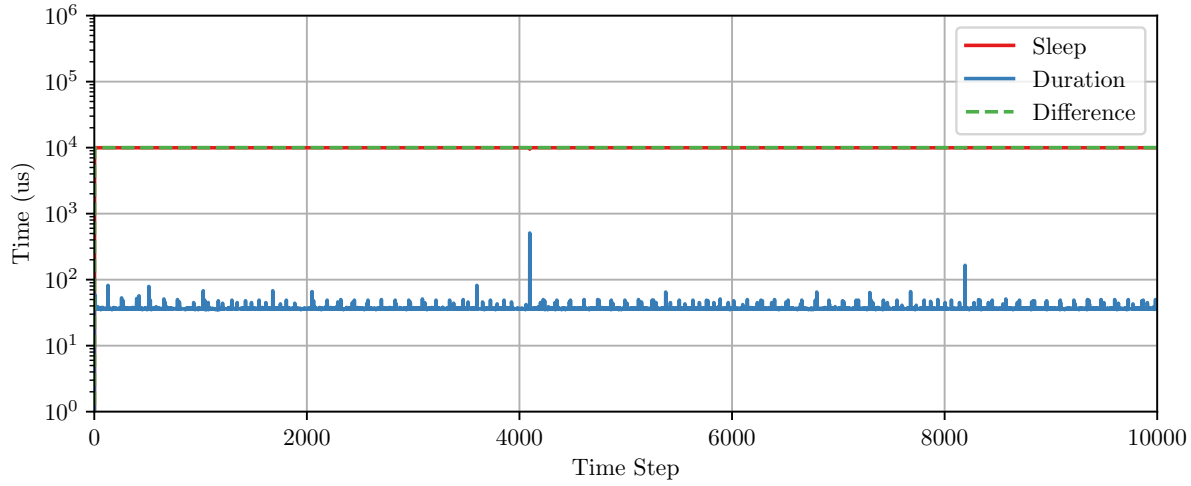


Figure 4.4: Shared Memory Loop Execution

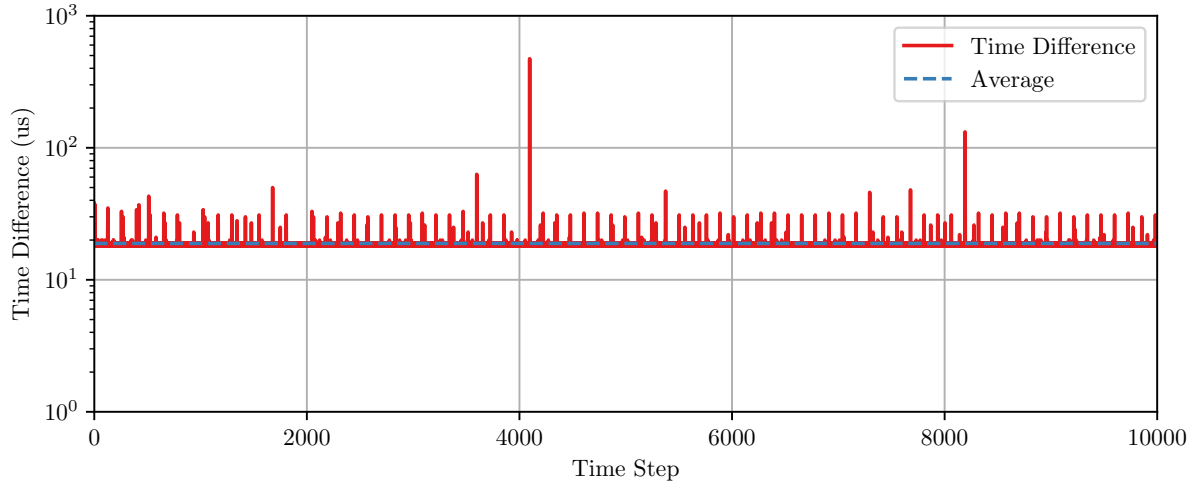


Figure 4.5: Shared Memory Execution Latency

4.4 Hardware

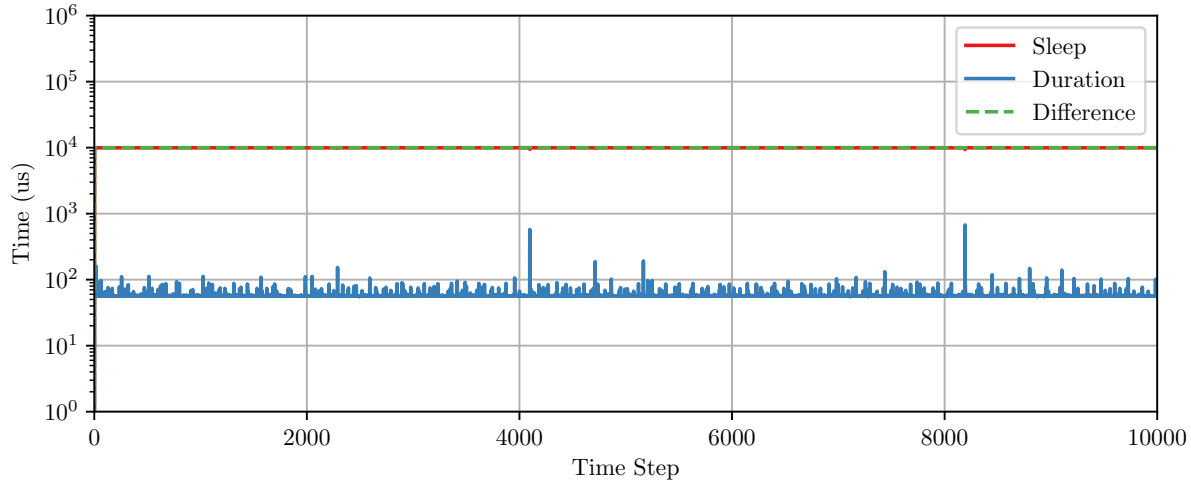


Figure 4.6: Socket Loop Execution

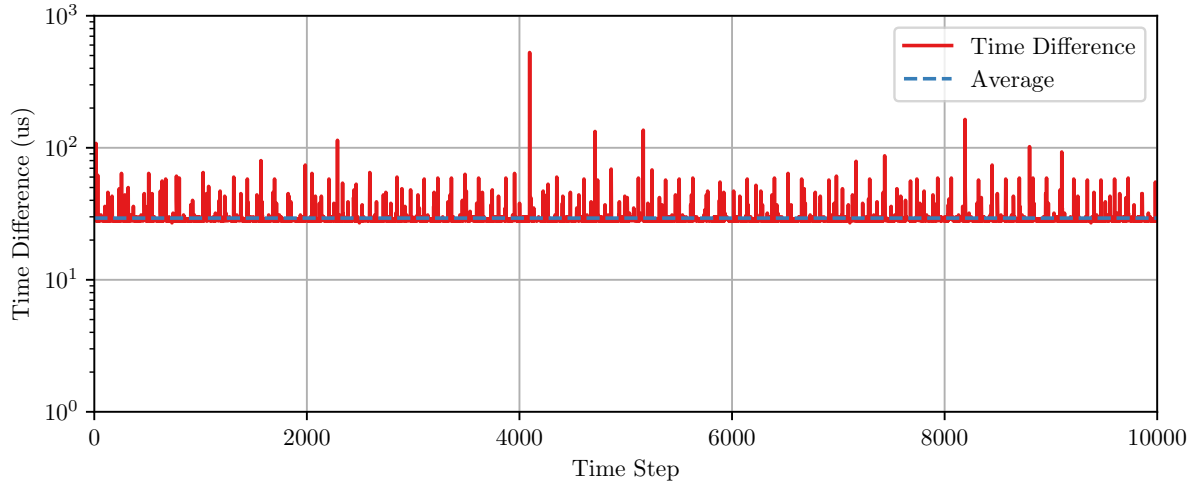


Figure 4.7: Socket Execution Latency

4.4.1 Test Harness

Though this harness, an example control loop implementation can be simulated to ensure all deadlines are met during a live software update.

Figure 4.9 contains a signal generator, this randomly produces a signal and outputs a packet. These packets are ingested by the software infrastructure where processing occurs. An output packet is subsequently generated (mirroring the input) and sent to a signal receiver, which captures this packet.

The signal generator and receiver output their current state over discrete digital signals so that a logic analyser can record the state of the system over time. Specifically the states can be compared to measure the complete system latency during standard operation and software update. If the generator and receiver produce additional reference clock

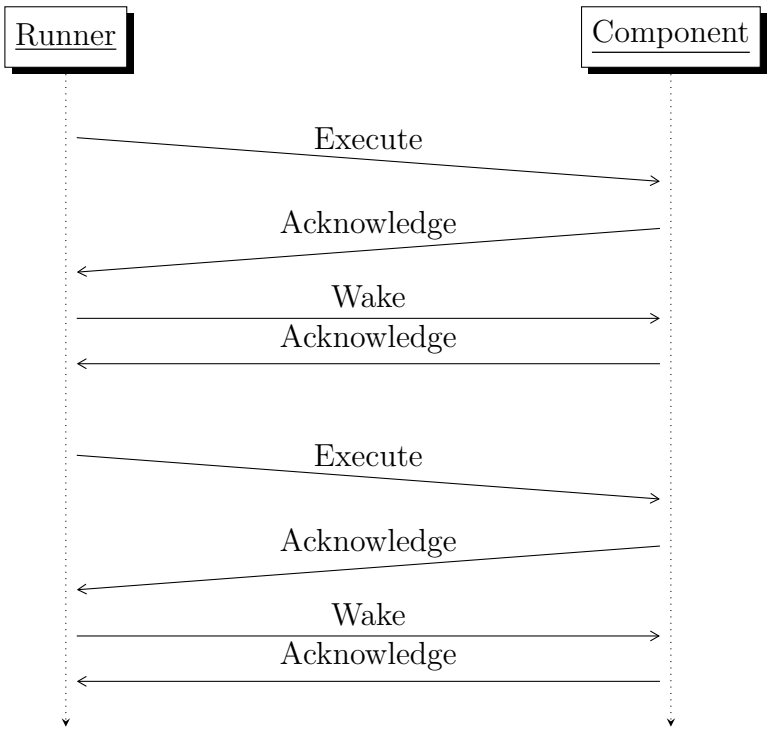


Figure 4.8: Client-Server messaging

signals, the logic analyser can more closely calculate the software infrastructure induced latency.

The generator, receiver and logic analyser were all implemented using the RP2040 microprocessor from Raspberry Pi for hardware simplification.

The two controllers that performed packet communication used the W5500 chip to send UDP packets over ethernet, this integrates nicely with Elafry messages. This co-processor offloads the complex ethernet drivers and communicates to the main proicessor over

This communicates over SPI and offload sthe compelx ethernet stuff onto this co-processor. SPI signals confiure the IP address of the taregt device and allow the user to send data.

The intention for this software was to also use Rust, however we could not get the Rust libaries for the W5500 working quickly, sure with extra time with sould have been possible. To simplify this software the arduino development environment was used. This as all the drivers for the w5500 are integrated in arduino libraries.

Logic analyser https://github.com/dotcypress/ula

Capture the status of the 16 input channels on the Pi Pico at specified frequencies. When integrated with teh Pulseview analysis software we can idenitify the latencies

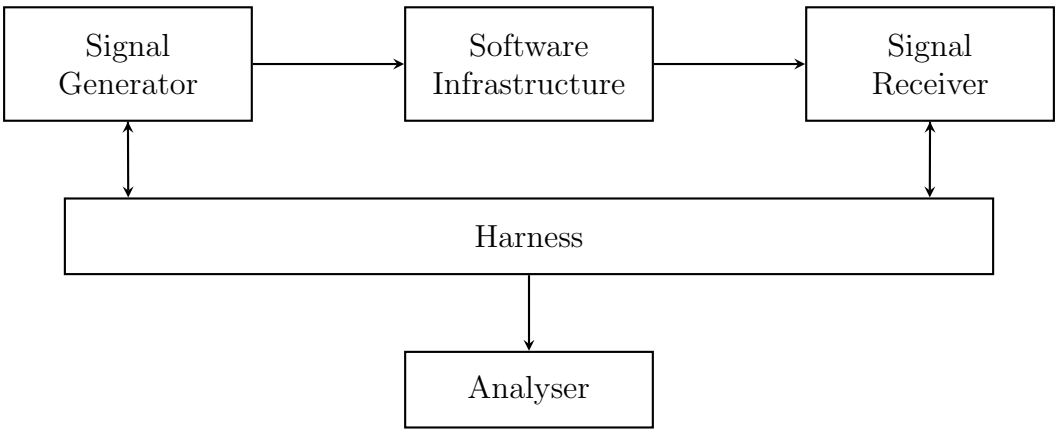


Figure 4.9: Test Harness Design

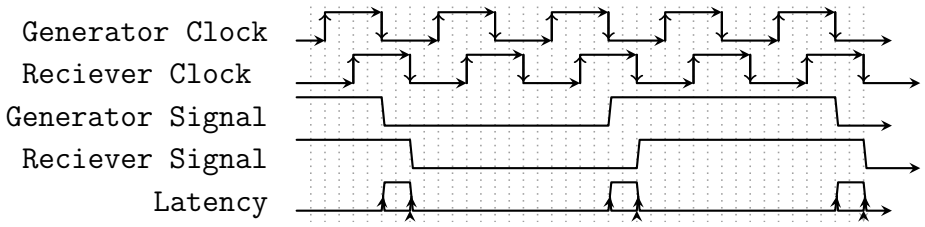


Figure 4.10: Test Harness Capture

4.4.2 Development Cards

In order to build and test the real time software, dedicated development hardware is required to ensure an isolated testing environment.

These test cards consist of a Raspberry Pi Compute Module 4 (CM4). This contains the same overall hardware as the standard raspberry pi 4. EMMC flash for high vibration environments. Exception being the ethernet IC and EMMC flash (This later caused problems with compiling the operating system and bootloader)

Key requirement is remote debugging, full remote power and data control. Also placed in a rack Selected the Pi KVM project for remote control as is supports deploy as well as additional GPIO control.

The CM4's display is captured using the CSI port on the KVM with a HDMI to CSI converter. The power is controlled using a relay, that controls the 12v supply to the CM4. The signal flags of the EMMC storage are directly connected to the KVM. EXPLAIN the signal flags These are configured in the KVM user interface UART is connected to a USB serial converter. USB host is connected to the KVM for flashing the CM4. Finally a debugger (JTAG) in case of modifications to the kernel itself.

Two of these so that multiple benchmarking activities can be performed at once and compared.

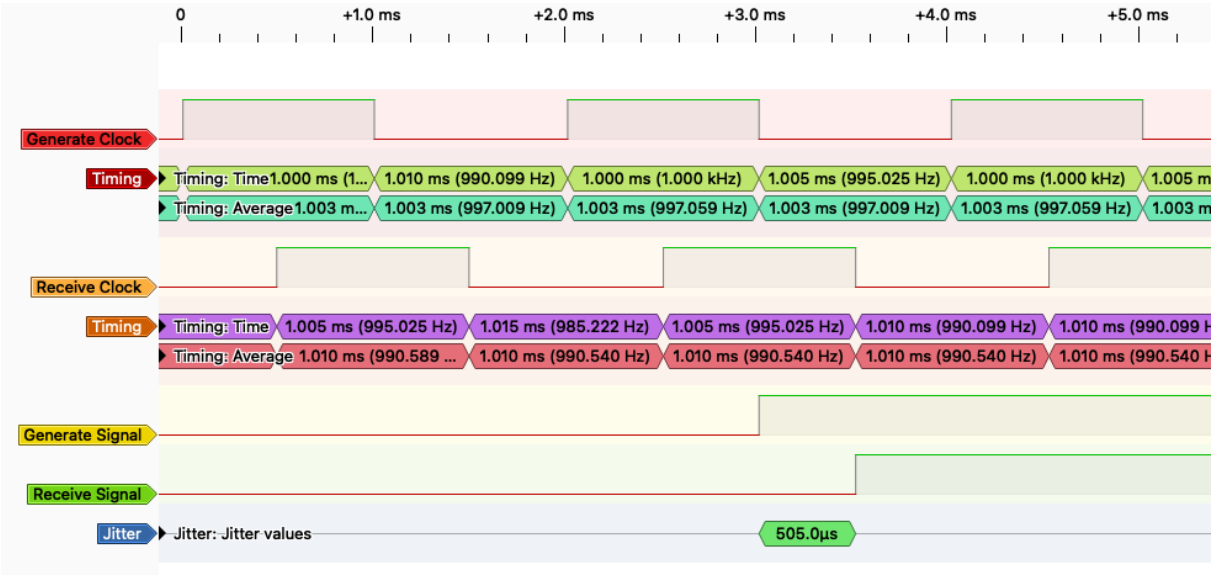


Figure 4.11: Test Harness PluseView Capture

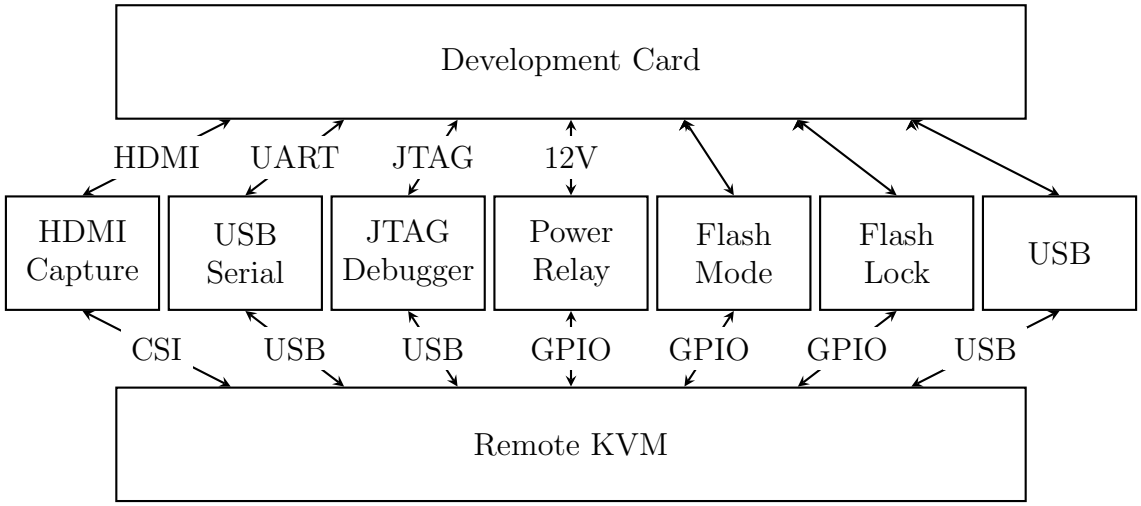


Figure 4.12: Development Hardware

Use the uBoot boot loader. This allows the development images to be loaded over TFTP, we have a dedicated tftp boot server.

Connected to a managed switch. The test harness is connected to this on a separate VLAN.

Node 1
Node 2

Fitting Node

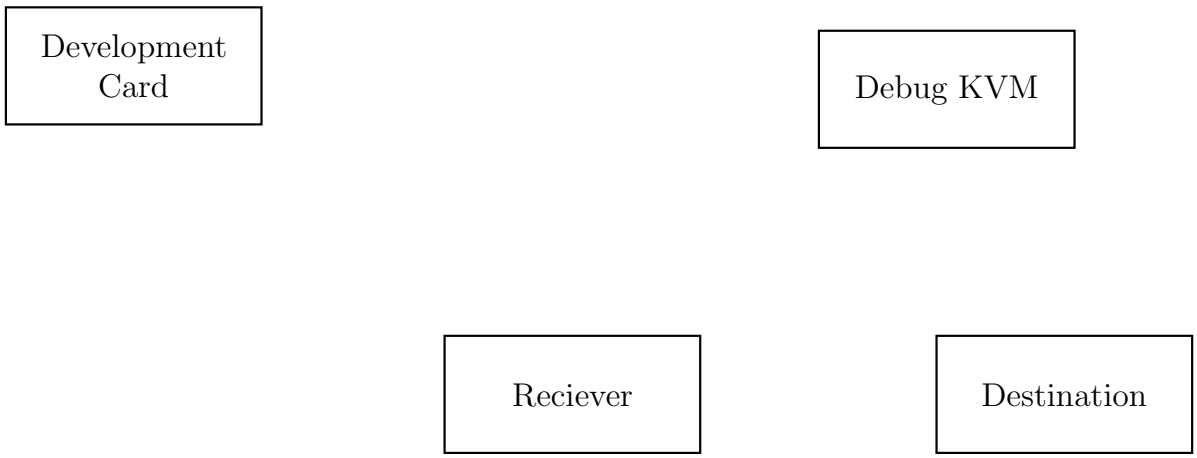
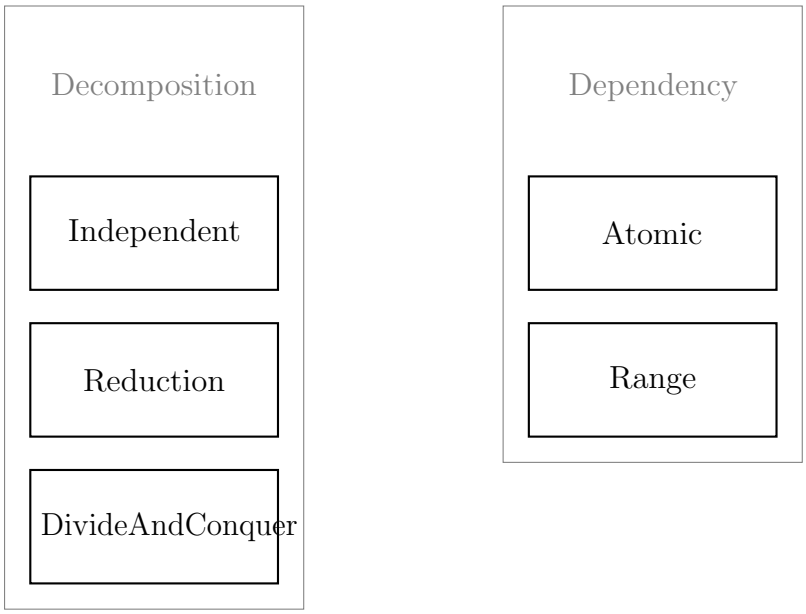


Figure 4.13: Heatmap for class distribution in dataset



Chapter 5

Implementation

Chapter 6

Testing

Chapter 7

Evaluation

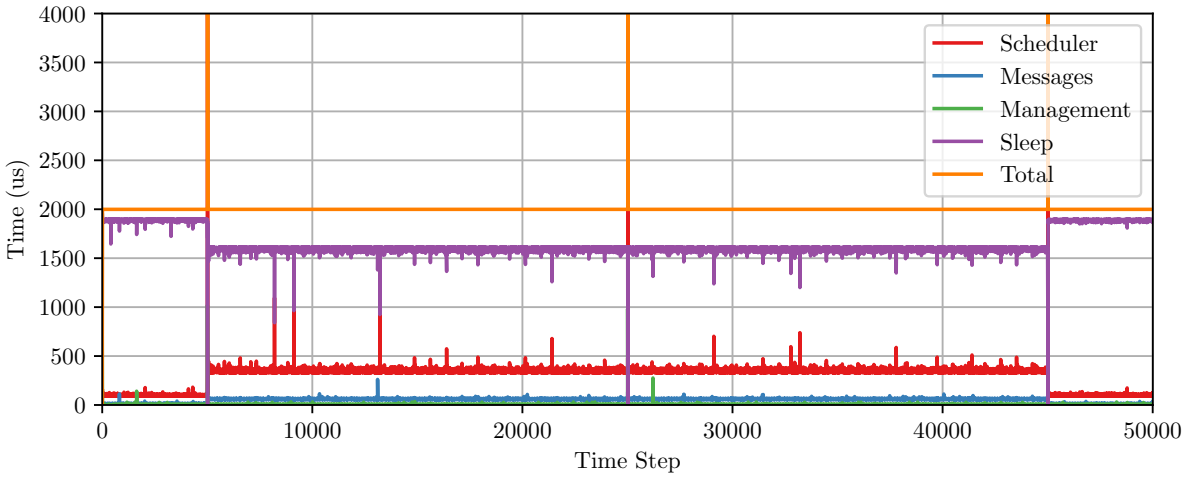


Figure 7.1: Elafry Demo Times

Chapter 8

Conclusion

References

Abeni, Luca and Dario Faggioli (June 2020). "Using Xen and KVM as Real-Time Hypervisors". In: *Journal of Systems Architecture* 106, p. 101709. ISSN: 13837621. DOI: 10.1016/j.sysarc.2020.101709. URL: https://linkinghub.elsevier.com/retrieve/pii/S1383762120300035 (visited on 10/06/2023).

Kavi, Krishna, Robert Akl, and Ali Hurson (Mar. 16, 2009). "Real-Time Systems: An Introduction and the State-of-the-Art". In: *Wiley Encyclopedia of Computer Science and Engineering*. Ed. by Benjamin W. Wah. 1st ed. Wiley, pp. 2369–2377. ISBN: 978-0-471-38393-2 978-0-470-05011-8. DOI: 10.1002/9780470050118.ecse344. URL: https://onlinelibrary.wiley.com/doi/10.1002/9780470050118.ecse344 (visited on 03/05/2024).

Stallings, W. (2018). *Operating Systems: Internals and Design Principles, Global Edition*. Pearson Education. ISBN: 978-1-292-21430-6. URL: https://books.google.co.uk/books?id=-76GEAAAQBAJ.

Appendix A

Extra Information

Some more text ...